

SKILL : WEEK 6

SESSION 11:

Task -1:

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

#define MAX 5
#define SIZE 50

int main()
{
    char history[MAX][SIZE];
    int count = 0;
    char command[SIZE];

    while (1)
    {
        printf("Enter command: ");
        fgets(command, SIZE, stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        // Store command
        if (count < MAX)
        {
            strcpy(history[count], command);
            count++;
        }
        else
        {
            // Remove oldest command
            for (int i = 1; i < MAX; i++)
                strcpy(history[i - 1], history[i]);

            strcpy(history[MAX - 1], command);
        }

        // Display history
        printf("\nHistory:\n");
        for (int i = 0; i < count; i++)
            printf("%d: %s\n", i + 1, history[i]);

        printf("\n");
    }

    // Retrieve last entry
    if (count > 0)
        printf("Last command: %s\n", history[count - 1]);

    // Validate consistency
    if (count <= MAX)
        printf("History is consistent.\n");

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week6$ nano history.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week6$ gcc history.c -o history
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week6$ ./history
Enter command: ls

History:
1: ls

Enter command: pwd

History:
1: ls
2: pwd

Enter command: cd /tmp

History:
1: ls
2: pwd
3: cd /tmp

Enter command: date

History:
1: ls
2: pwd
3: cd /tmp
4: date

Enter command: whoami

History:
1: ls
2: pwd
3: cd /tmp
4: date
5: whoami

Enter command: uname

History:
1: pwd
2: cd /tmp
3: date
4: whoami
5: uname

Enter command: exit
Last command: uname
History is consistent.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week6$ █
```

OBSERVATION:

The program successfully implemented a command history buffer to store and maintain previously entered commands. The history was displayed after each command, allowing the stored entries to be retrieved and reviewed. A fixed capacity of five commands was maintained, and when the buffer became full, the oldest command was removed to accommodate the newest entry. The program also successfully retrieved the last stored command and verified that the history remained within its defined capacity. Thus, command storage, history maintenance, capacity handling, entry retrieval, history display, and consistency validation were successfully demonstrated.

Task -2:

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

CODE:

```
GNU nano 2.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    int pipefd[2];
    pid_t p1, p2;
    char *cmd1[] = {"ls", NULL};
    char *cmd2[] = {"wc", "-l", NULL};

    // Create pipeline
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    // First command
    p1 = fork();

    if (p1 == 0)
    {
        close(pipefd[0]);
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execvp(cmd1[0], cmd1);
        perror("exec");
        return 1;
    }
}
```

```
}

// Second command
p2 = fork();

if (p2 == 0)
{
    close(pipefd[1]);
    dup2(pipefd[0], STDIN_FILENO);
    close(pipefd[0]);

    execvp(cmd2[0], cmd2);
    perror("exec");
    return 1;
}

// Parent
close(pipefd[0]);
close(pipefd[1]);

waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);

printf("Pipeline executed successfully.\n");

return 0;
```

OUTPUT:

```
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week6$ nano pipeline.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week6$ gcc pipeline.c -o pipeline
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week6$ ./pipeline
4
Pipeline executed successfully.
```

OBSERVATION:

The program successfully implemented a pipeline structure using the `pipe()` system call and created separate processes using `fork()`. The first child process executed the `ls` command and redirected its output to the pipe using `dup2()`. The second child process received this output through the pipe and executed the `wc -l` command. The execution order and communication between the processes were maintained correctly, and `waitpid()` was used by the parent process to synchronize with both child processes. The successful output confirmed that the pipeline was correctly connected and executed. Thus, pipeline creation, command storage, execution order, process communication, pipeline validation, and testing were successfully demonstrated.

SESSION 12 :

Task -1 :

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t p1, p2;

    // Create pipe
    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    // First process: ls
    p1 = fork();

    if (p1 == 0)
    {
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);

        execlp("ls", "ls", NULL);
        perror("exec");
        return 1;
    }

    // Second process: wc -l
    p2 = fork();

    if (p2 == 0)
    {
        close(fd[1]);

        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);

        execlp("wc", "wc", "-l", NULL);
        perror("exec");
        return 1;
    }
}
```

```
// Parent closes both descriptors
close(fd[0]);
close(fd[1]);

// Synchronize processes
waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);

printf("Data flow completed successfully.\n");

return 0;
}
```

OUTPUT:

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Skill/Week6
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week6$ nano pipe.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week6$ gcc pipe.c -o pipe
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week6$ ./pipe
6
Data flow completed successfully.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week6$ |
```

OBSERVATION:

The program successfully created a pipe to establish communication between two processes. The first child process executed the `ls` command and redirected its standard output to the pipe using `dup2()`. The second child process received the data from the pipe through its standard input and executed the `wc -l` command to process the received data. The unused pipe descriptors were properly closed in both child processes and the parent process. The parent used `waitpid()` to synchronize with both child processes and ensure that they completed execution before the parent terminated. The successful output confirmed that data flowed correctly from the first command to the second command. Thus, pipe creation, stream redirection, connected command execution, descriptor management, process synchronization, and data flow were successfully demonstrated.

Task - 2:

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2][2];
    pid_t pid[3];

    // Create two pipes
    pipe(fd[0]);
    pipe(fd[1]);

    // First process: ls
    pid[0] = fork();

    if (pid[0] == 0)
    {
        dup2(fd[0][1], STDOUT_FILENO);

        close(fd[0][0]);
        close(fd[0][1]);
        close(fd[1][0]);
        close(fd[1][1]);

        execlp("ls", "ls", NULL);
        return 1;
    }

    // Second process: grep
    pid[1] = fork();

    if (pid[1] == 0)
    {
        dup2(fd[0][0], STDIN_FILENO);
        dup2(fd[1][1], STDOUT_FILENO);

        close(fd[0][0]);
        close(fd[0][1]);
        close(fd[1][0]);
        close(fd[1][1]);

        execlp("grep", "grep", ".c", NULL);
        return 1;
    }
}
```

```
// Third process: wc
pid[2] = fork();

if (pid[2] == 0)
{
    dup2(fd[1][0], STDIN_FILENO);

    close(fd[0][0]);
    close(fd[0][1]);
    close(fd[1][0]);
    close(fd[1][1]);

    execlp("wc", "wc", "-l", NULL);
    return 1;
}

// Parent closes all pipe descriptors
close(fd[0][0]);
close(fd[0][1]);
close(fd[1][0]);
close(fd[1][1]);

// Wait for all processes
for (int i = 0; i < 3; i++)
    waitpid(pid[i], NULL, 0);

printf("Long pipeline executed successfully.\n");

return 0;
}
```

OUTPUT:

```
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week6$ nano multi_pipe.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week6$ gcc multi_pipe.c -o multi_pipe
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week6$ ./multi_pipe
4
Long pipeline executed successfully.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week6$ |
```

OBSERVATION:

The program successfully implemented a long pipeline using two pipes and three processes. The `ls` output was passed to `grep ".c"` through the first pipe, and the filtered output was passed to `wc -l` through the second pipe. The parent process used `waitpid()` to synchronize all child processes, while unused file descriptors were closed for proper resource cleanup. Thus, multiple pipes, process coordination, descriptor management, and data flow were successfully demonstrated.